Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Analyzing list and dictionary operations

Today
Welcome to your next lesson: Analyzing list and dictionary operations.

I'm preparing your personalized lesson based on what you've already learned, what's coming up next, and how to help you lock in your skills.

Keep it up, every step counts.

Imagine two things in a game 🎮:

Your inventory: an ordered list of items (slot 0: sword, slot 1: shield, etc.) → this feels like a Python list.
Your stats by name: "hp", "mana", "strength" → you don’t care about order, you care about names → this feels like a Python dict.

We’re going to compare how fast different actions are on these two: lists and dicts.

To warm up:

If you already know the exact position of an item (like “slot 3 in inventory”), which structure do you think is a better natural fit?

A Python list
A Python dictionary

Just answer in words; no need to explain yet.

Python list

Nice, I agree — a Python list is a natural fit when you care about positions like “slot 3” ☑

Let’s zoom in on just one operation: indexing in a list.

If you have:

Python

items = ["sword", "shield", "potion", "bow"]
x = items[2]
Getting items[2] (or any items[i]) takes constant time, written as O(1) — it's basically "jump straight to that slot", no matter how long the list is.

Does this idea make sense:

"Access by index in a list is O(1) because we jump straight to the position, not walk through the list"?

Answer yes/no in your own words, then we'll add the next list operation.

Yes, that makes sense

Great 👍

Next, still with lists, let's look at appending.

In Python:

Python

items = ["sword", "shield"]
items.append("potion")
Adding at the end with append is treated as O(1) time (often called amortized O(1)): each append is basically "place one more thing at the end", and the cost doesn't grow with the list size in a way we worry about here.

So, so far for lists:

Indexing items[i] → O(1)
Appending at the end items.append(x) → O(1) (amortized)

Ready to look at a slower list operation (where the cost does grow with the list size)?

Yes, show me the slower operation

Nice 🔥 let's slow it down and look at inserting in the middle of a list.

Picture a row of cards on a table:

Text

[A][B][C][D]
You want to insert X between B and C:

You place X there
But now you must shift C and D one step to the right to make room

More items ⇒ more shifting.

In Python:

Python

items = ["A", "B", "C", "D"]
items.insert(2, "X") # insert at index 2
This "shifting everyone to the right" makes insert in the middle O(n) time — cost grows with the list length.

Same story for pop(i) or del items[i] in the middle: remove one, then shift everything

after it left → O(n).

So for lists:

items[i] → O(1)
append(x) → O(1) (amortized)
insert(i, x) or pop(i) in the middle → O(n)

Does this O(n) because of “shifting many elements” land clearly for you, or want one more concrete example with numbers?

its clear

Perfect 👍

Let’s switch to dictionaries for a moment.

Think of a phone book in your phone:

Python

```
contacts = {
    "Alice": "555-1111",
    "Bob": "555-2222",
    "Carol": "555-3333"
}

number = contacts["Bob"]
```

Here you don’t say “give me contact #1”, you say “give me the contact named 'Bob'”.

For a Python dict, this kind of lookup by key:

Python

```
contacts["Bob"]
```

is usually O(1) time on average — it’s like jumping straight to Bob’s entry without scanning everyone.

We’ll ignore the rare worst-case for now and treat dict key lookup as “basically constant time”.

So far:

List index lookup items[i] → O(1)
Dict key lookup contacts["Bob"] → O(1) on average

Want to see how inserting into a dict compares to inserting into a list?

Yes, compare the insertions

Great, let’s line them up side by side 👀

For a list:

items.insert(i, x) in the middle/start → many elements must shift right → O(n)

For a dict:

Python

```
stats = {}
```

```
stats["hp"] = 100      # insert new key
stats["mana"] = 50     # insert another key
Inserting a new key like "hp" or "mana" is treated as O(1) on average
Updating an existing key, e.g. stats["hp"] = 120, is also O(1) on average

So:


List insert at position → O(n) (because of shifting)
Dict insert by key → O(1) average (no shifting through a bunch of positions)
Quick check before we move on:


If you need to add 10,000 new stats by name ("hp", "mana", "strength", …) and then often look them up by name, which sounds better for speed: a list of (name, value) pairs or a dictionary from name -> value? Just answer “list” or “dict”.

dictionary

Exactly — a dictionary is the right call there 💯


Let’s pull the pieces together with a simple rule of thumb 🤗


Use a list when you care about order/positions

Fast: items[i], append(x) → O(1)
Slow: insert/remove in the middle → O(n)
Use a dict when you care about names/keys

Fast (average): d[key], d[key] = value, key in d → O(1)
You don’t worry about positions at all
I’d like you to try a tiny coding exercise next:


In the code editor, write two versions of a “phone book”:


A list of pairs, like [("Alice", "555-1111"), ...], and “find Bob’s number” by scanning the list
A dict like {"Alice": "555-1111", ...}, and “find Bob’s number” by direct key lookup

For each version, add print() statements that show how many checks you did (steps).


Go ahead and implement that in the editor and Submit code. I’ll review it and we’ll connect the behavior back to the time complexity.

Python

phone_list = [("Alice", "555-1111"), ("Bob", "555-2222"), ("Tom", "555-3333")]
phone_dict = {"Alice": "555-1111", "Bob": "555-2222", "Tom": "555-3333"}
i =  1
bobs_number = None
for phone in phone_list:
    if phone[0] == "Bob":
        bobs_number = phone[1]
    print(f'Pass {[i]} on list')
    i += 1
    if bobs_number:
        break

i = 1
bobs_numeber = phone_dict["Bob"]
print(f'Pass {[i]} on dict')
Nice work, Ron 👏 Your code does exactly what I was hoping for.


```

Let’s read what it’s showing:

List version:
Printed:
Pass [1] on list
Pass [2] on list
That means it had to check 2 entries in the list before it found "Bob" and broke.
Dict version:
Printed:
Pass [1] on dict
That’s 1 direct step to get "Bob".

This matches the Big O idea:

List search for Bob → might need many checks → O(n)
Dict lookup phone_dict["Bob"] → one direct jump → O(1) average
I want to lock in your understanding with a quick visual comparison next.

Would you like me to show you a small table comparing common list vs dict operations and their time complexities?

Yes, show the table

Here’s a clean side-by-side view for the operations we’ve talked about 📊

Common Python operations and their time complexity

Structure	Operation	Example code	Time complexity
List	Index by position	x = items[i]	O(1)
List	Append at end	items.append(x)	O(1) (amortized)
List	Insert/remove at arbitrary position	items.insert(i, x) / items.pop(i)	O(n)
List	Search by value (e.g. find "Bob")	loop & compare	O(n)
Dict	Lookup by key	x = d[key]	O(1) (avg)
Dict	Insert or update by key	d[key] = value	O(1) (avg)
Dict	Membership test by key	key in d	O(1) (avg)

Simple rule:

Need order/positions → think list
Need fast access by name/key → think dict
Want to do one quick multiple□choice question to check you’ve got when to pick list vs dict?

Yes, give me the question

What do you think? 👇

You are building a system where you will repeatedly look up users by their username (a string like "ron_h") to get their profile. You almost never care about order, only fast lookups by username. Which data structure is the better fit for speed?

A.

A Python list of (username, profile) tuples that you scan each time

B.

A Python dictionary mapping username to profile

Nice job, Ron 🎉 You picked the dictionary mapping username → profile, which is exactly right.

Why this wins:

With a dict, profiles["ron_h"] is O(1) average time — a direct jump to the user.
With a list of tuples, you'd have to scan through each pair until you find the right username → O(n) time.

So for "look up by username, over and over, order doesn't matter", a dict is the perfect fit 🚀

At this point you've:

Matched list vs dict to the right use cases
Understood O(1) vs O(n) for the core operations

Do you feel ready to wrap up this lesson and move to the next one, or do you want one more practice scenario first?

I'm ready for the next lesson

Love it. You're moving fast and solid, Ron ⚔️

Quick recap of what you've just mastered:

Lists
items[i] and append(x) → O(1) (indexing, appending at end)
Insert/remove in the middle, or search by value → O(n) because many elements may shift or be checked
Dicts
d[key], d[key] = value, key in d → O(1) on average (fast direct access by key)
Choosing structure
Care about positions / order → use a list
Care about fast lookups by name/key → use a dict
You didn't just memorize this — you used it in code and decisions. That's exactly how you build real skill 💪

When you're ready, go ahead and hit Next Lesson in the Learn tab, and we'll keep stacking on this.

```
phone_list = [("Alice", "555-1111"), ("Bob", "555-2222"), ("Tom", "555-3333")]
phone_dict = {"Alice": "555-1111", "Bob": "555-2222", "Tom": "555-3333"}
i =  1
bobs_number = None
for phone in phone_list:
    if phone[0] == "Bob":
        bobs_number = phone[1]
    print(f'Pass {[i]} on list')
    i += 1
    if bobs_number:
        break

i = 1
bobs_numeber = phone_dict["Bob"]
print(f'Pass {[i]} on dict')
```